Cache Warming

1. What Is Cache Warming?

Cache warming means preloading cache entries **before real users request them**, so first requests are cache hits instead of cache misses.

Instead of:

```None
User request → Cache miss → DB query → Slow response
```

Use:

```None
Preload cache first

User request → Cache hit → Fast response
```

2. Where It Applies

Used in shared caching systems like:

```None
CDN

Redis

Memcached

API Gateway Cache

Backend Cache Layer
```

Usually **not** for browser cache.

3. Why Use Cache Warming?

Without warming:

```None
First user gets slower response

Later users get fast responses
```

With warming:

```None
Even first user gets fast response
```

Useful for high-traffic systems.

4. Example

E-commerce sale starts at 10:00 AM.

Popular pages:

```None
iPhone page

Laptop page

TV offers page
```

Warm cache at 9:55 AM.

Then traffic spike hits:

```None
All users get cached responses

Lower DB pressure
```

5. Common Ways to Warm Cache

A. Scheduled Preload

Run job before known traffic spike.

```None
Every morning 8 AM preload homepage
```

B. Deployment Warmup

After restart / deploy:

```None
Load hot keys into Redis
```

C. Predictive Warming

Use analytics:

```None
Top 1000 most visited products

Top cities searched

Trending content
```

Preload only hot data.

D. Background Crawlers

Bots simulate requests:

```None
GET homepage

GET category pages

GET top articles
```

6. Advantages

```None

Fast first request

Better user experience

Reduced cold start latency

Lower DB spikes during launch

Higher cache hit ratio early
```

7. Major Disadvantages

A. Complexity

Large cache clusters may have:

```None
Thousands of nodes

Millions of keys
```

Warming becomes operationally hard.

B. Extra Traffic

To warm cache, system must fetch data from:

```None
Frontend

Backend

Database

Other APIs
```

Can create load before users even arrive.

C. Wasteful for Rare Data

Many preloaded keys may never be used.

```None
Unused cache entries waste memory
```

D. TTL Problems

If TTL short:

```None
Entries expire before users request them
```

If TTL long:

```None
Stale data risk

Higher memory cost
```

8. Better Strategy Often = Partial Warming

Instead of warming everything:

```None
Warm top 1% hot keys only
```

Examples:

```None
Top products

Homepage widgets

Popular search results

Trending videos
```

Most practical solution.

9. Real-World Examples

CDN

Warm static assets:

```None
JS bundles

CSS

Images
```

Streaming Platform

Warm:

```None
Trending shows

Popular thumbnails

Landing pages
```

E-commerce

Warm:

```None
Sale catalog

Top brands

Best seller
```

10. Cold Start vs Warm Cache

Cold cache:

```None
High latency

Low hit ratio

DB pressure
```

Warm cache:

```None
Low latency

Immediate hits

Stable backend
```

11. Important Engineering Principle

Sometimes easier than warming cache:

```None
Make cache misses fast enough
```

If uncached P99 is already low:

```None
No need for expensive warming system
```

12. Interview Talking Point

If asked when to use:

```None
Use cache warming for launches, daily traffic spikes,
deployments, hot-content systems.
```

If not needed:

```None
Use lazy loading + good DB performance.
```

13. Best HLD Recommendation

```None
Warm only hot keys

Rate-limit warm jobs

Run after deployments

Use analytics-driven preload

Fallback to lazy cache fill
```

14. One-Line Summary

Cache warming means preloading popular data into shared caches before users request it, improving first-request latency and reducing backend load, but it adds complexity, cost, and possible wasted cache space.